

PROGRAMMATION FONCTIONNELLE 2 : TD4

février 2026 — Pierre Rousselin

Exercice 1 : Quelques fonctions génériques standard en OCaml

Pour chacune des fonctions suivantes, donner des définitions (une avec des arguments avant `=`, une autre sans) et un type en OCaml.

1. La fonction `ignore` ignore son unique argument et a une valeur de type `unit`.
2. La fonction `const c` est une fonction constante égale à `c`.
3. La fonction `compose` est une fonction qui compose deux fonctions données en argument (`compose f g` correspond en mathématiques à $f \circ g$).
4. La fonction `flip` prend une fonction à deux arguments en argument et retourne la même fonction avec ses arguments dans l'autre ordre.
5. L'opérateur `|>` applique une fonction (son second argument) à un argument (son premier argument). La définition d'un tel opérateur s'écrit en commençant par

```
let ( |> ) = ...
```
6. L'opérateur `@@` applique une fonction (son premier argument) à un argument (son second argument).
7. Les opérateurs (binaires infixes) d'OCaml sont des suites de symboles qui obéissent à des conventions (assez arbitraires) qu'on peut trouver dans la documentation officielle. En particulier,
 - les opérateurs (binaires infixes) qui commencent par le symbole `|` sont très peu prioritaires et associatifs à gauche
 - les opérateurs (binaires infixes) qui commencent par le symbole `@` sont à la priorité juste au-dessus (donc peu prioritaires) et associatifs à droite.

a) Réécrire la définition suivante en n'utilisant aucune parenthèse avec l'opérateur `@@` :

```
let res = let x = 3.14 in sqrt (cos (sin (tan (2. +. x))))
```

b) Idem avec `|>`.

--- * ---

Correction de l'exercice 1 :

```
let ignore : 'a -> unit = fun _ -> ()
let ignore_sugared _ = ()
let const : 'a -> 'b -> 'a = fun c -> fun _ -> c
let const_sugared c _ = c
let compose : ('b -> 'c) -> ('a -> 'b) -> ('a -> 'c) =
  fun f -> fun g -> fun x -> f (g x)
let compose_sugared f g x = f (g x)
let flip : ('a -> 'b -> 'c) -> ('b -> 'a -> 'c) =
  fun f -> fun x -> fun y -> f y x
let flip_sugared f x y = f y x
let ( |> ) : 'a -> ('a -> 'b) -> 'b = fun x -> fun f -> f x
let pipe_sugared x f = f x
let ( @@ ) : ('a -> 'b) -> 'a -> 'b = fun f -> fun x -> f x
let app_sugared f x = f x
let res = let x = 3.14 in sqrt (cos (sin (tan 2. +. x)))
let res1 = let x = 3.14 in sqrt @@ cos @@ sin @@ tan @@ 2. +. x
let res2 = let x = 3.14 in 2. +. x |> tan |> sin |> cos |> sqrt
```

--- * ---

Exercice 2 : Quelques preuves de forall

Dans chacun des cas suivants, donner deux preuves : l'une avec un seul `exact`, l'autre avec des tactiques. On rappelle la définition de `and` et celle de `True` :

```
Inductive and (A B : Prop) : Prop := conj (hA : A) (hB : B).
Inductive True : Prop := I.
```

1. Lemma foo1 : forall (P Q : Prop), P -> Q -> P.
2. Lemma foo2 : forall (P Q : Prop), P -> Q -> P /\ Q.
3. Lemma foo3 :
forall (P : Prop), (forall (Q : Prop), Q -> P) -> P.

--- * ---

Correction de l'exercice 2 :

```
Lemma foo1 : forall (P Q : Prop), P -> Q -> P.
Proof.
  exact (fun P Q hP _ => hP).
Qed.
```

```
Lemma foo1' : forall (P Q : Prop), P -> Q -> P.
Proof.
  intros P Q hP _ . exact hP.
Qed.
```

```
Lemma foo2 : forall (P Q : Prop), P -> Q -> P /\ Q.
Proof.
  exact (fun P Q hP hQ => conj hP hQ).
Qed.
```

```
Lemma foo2' : forall (P Q : Prop), P -> Q -> P /\ Q.
Proof.
  intros P Q hP hQ. split; assumption.
Qed.
```

```
Lemma foo3 :
  forall (P : Prop), (forall (Q : Prop), Q -> P) -> P.
Proof.
  exact (fun P h => h True I).
Qed.
```

```
Lemma foo3' :
  forall (P : Prop), (forall (Q : Prop), Q -> P) -> P.
Proof.
  intros P h.
  specialize (h True).
  apply h.
  exact I.
Qed.
```

--- * ---

Exercice 3 : Quelques types dépendants en Rocq

1. Pour chacun des types inductifs suivants, donner le type complet de l'inductif et ceux des constructeurs.
 - a) Inductive foo (A B : Type) : Type :=
Foo1 (a : A) | Foo2 (a : A) (b : B).
 - b) Inductive bar (A B : Prop) : Prop :=
Barn | Bar1 (a : A) | Barr (b : B).

2. Le type `ex` est utilisé pour formaliser le quantificateur « il existe » ($\exists$) en Rocq. Voici sa définition :
`Inductive ex (A : Type) (P : A -> Prop) : Prop := ex_intro (x : A) (h : P x).`
 Donner son type et le type de son constructeur.
3. De quoi avons-nous besoin pour prouver un « il existe » ?

--- * ---

Correction de l'exercice 3 :

- 1.a) Le type de `foo` est `Type -> Type -> Type`. Le type de `Foo1` est

$$\forall (A B : \text{Type}), \quad A \rightarrow \text{foo } A B.$$

Le type de `Foo2` est

$$\forall (A B : \text{Type}), \quad A \rightarrow B \rightarrow \text{foo } A B.$$

- b) Le type de `bar` est `Prop -> Prop -> Prop`. Le type de `Barn` est

$$\forall (A B : \text{Prop}), \quad \text{bar } A B.$$

Le type de `Bar1` est

$$\forall (A B : \text{Prop}), \quad A \rightarrow \text{bar } A B.$$

Le type de `Barr`

$$\forall (A B : \text{Prop}), \quad B \rightarrow \text{bar } A B.$$

2. Le type de `ex` est

$$\forall (A : \text{Type}), (A \rightarrow \text{Prop}) \rightarrow \text{Prop}.$$

Le type de `ex_intro` est

$$\forall (A : \text{Type})(P : A \rightarrow \text{Prop})(x : A), \quad Px \rightarrow \text{ex } A P.$$

3. Pour construire une preuve de « il existe `x` de type `A` tel que `P x` », on doit fournir un certain `x` de type `A` ainsi qu'une preuve (un terme) de `P x`.

--- * ---